

武汉理工大学

课 程 名 称	企 业 级 应 用 软 件 设 计 与 开 发
项 目 名 称	个 体 电 商 后 台 运 营 辅 助 系 统
方 向	企 业 级 应 用 软 件 的 A g e n t 改 造
学 号	2 0 2 5 3 0 3 0 2 2
姓 名	魏 嘉 均
专 业	软 件 工 程
指 导 教 师	戚 欣
提 交 日 期	2 0 2 6 年 6 月 2 2 日

一、选题背景与设计思想

1.1 问题定义

在电子商务行业高速发展的背景下，企业每日产生的业务数据量呈现指数级增长，尤其对于个体电商经营者而言，日常运营中需要投入大量时间对销售数据进行整理、分析与解读。然而，传统的人工数据分析模式在实际应用中暴露出若干显著缺陷。首先，人工处理报表的方式效率极为低下，不仅耗时费力，而且难以对业务环境的变化做出实时响应。其次，异常情况的发现机制存在明显滞后性，依赖于人工巡检的方式使得许多潜在问题无法在第一时间被识别和处理。最后，由于业务数据分散存储于不同的系统模块之中，决策者难以形成全局视角，导致关键决策缺乏充分的数据支撑。

痛点	示例
效率低下	人工分析报表耗时费力，无法实时响应业务变化
异常发现滞后	依赖人工巡检，异常情况难以及时发现
决策缺乏数据支撑	数据分散在各个系统，难以形成全局视角

1.2 现有方案不足

与此同时，传统电商后台系统在功能设计上存在三方面的主要局限。第一，系统采用被动式数据查看模式，用户必须主动发起查询请求才能获取所需数据，系统本身缺乏主动推送有价值信息的能力。第二，报表生成采用固定格式，内容仅限于对当日数据的常规统计分析，无法提供更深层次的智能分析结论和业务洞察。第三，异常预警机制较为薄弱，仅能支持基于简单阈值的告警规则，缺乏由人工智能驱动的智能检测能力，难以应对复杂多变的业务异常场景。

1.3 项目价值

为克服上述问题，本项目致力于构建一个基于智能体技术的自动化数据分析平台，该平台具备三项核心功能。在自动化报表生成方面，系统能够按照预设时间周期自动生成业务报表，从而将人力资源从重复性劳动中解放出来。在智能异常检测方面，平台利用人工智能技术实现业务异常的自动识别，能够提前发现潜在问题并发出预警。在自然语言分析方面，系统借助大语言模型驱动数据分析过程，以自然语言形式向用户呈现业务洞察。此外，平台还提供丰富的可视化图表展示功能，使用户能够直观掌握业务发展趋势。

功能	示例
自动化报表生成	定时自动生成业务报表，解放人力
智能异常检测	基于 AI 的异常识别，提前发现潜在问题
自然语言分析	LLM 驱动的数据分析，用自然语言呈现洞察
可视化展示	丰富的图表展示，直观呈现业务趋势

1.4 技术路线

本项目所采用的技术路线如图 1.1 所示，整体流程涵盖数据采集、数据预处理、特征工程、智能体调度、大语言模型调用、结果生成与可视化展示等多个环节，形成完整的数据分析闭环。

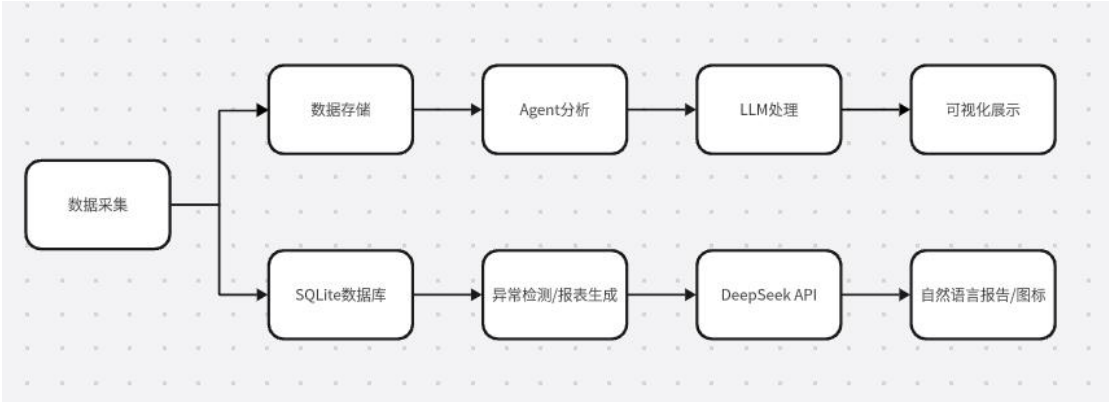


图 1.1 技术路线图

二、Specs 规格文档

2.1 Product Spec

在产品功能规格方面，本系统共规划六个核心功能模块。销售数据分析模块作为最高优先级功能，支持多维度的销售趋势分析以及各类别商品的销售占比统计。异常检测预警模块同样设定为 P0 优先级，负责实现库存异常与销量异常的智能检测，并根据严重程度进行分级告警。智能报表生成模块为 P0 优先级，能够自动生成日报与周报，并以自然语言形式呈现分析结论。库存管理模块列为 P1 优先级，提供库存记录的增删改查操作以及按类别筛选的功能。商品管理模块为 P1 优先级，实现商品信息管理及其与库存数据的关联维护。智能问答模块作为 P2 优先级，支持用户针对报表内容进行自然语言问答，并引入缓存机制以优化响应性能。

功能模块	功能描述	优先级
销售数据分析	多维度销售趋势分析、分类占比统计	P0
异常检测预警	库存/销量异常检测、分级告警	P0
智能报表生成	自动生成日报/周报、自然语言报告	P0
库存管理	库存 CRUD 操作、类别筛选	P1
商品管理	商品管理、库存关联	P1
智能问答	报表内容问答、缓存优化	P2

2.2 Architecture Spec

系统架构规格如图 2.1 所示，整体采用分层架构设计，各层之间通过明确定义的接口进行通信，确保了系统的模块化程度与可维护性。

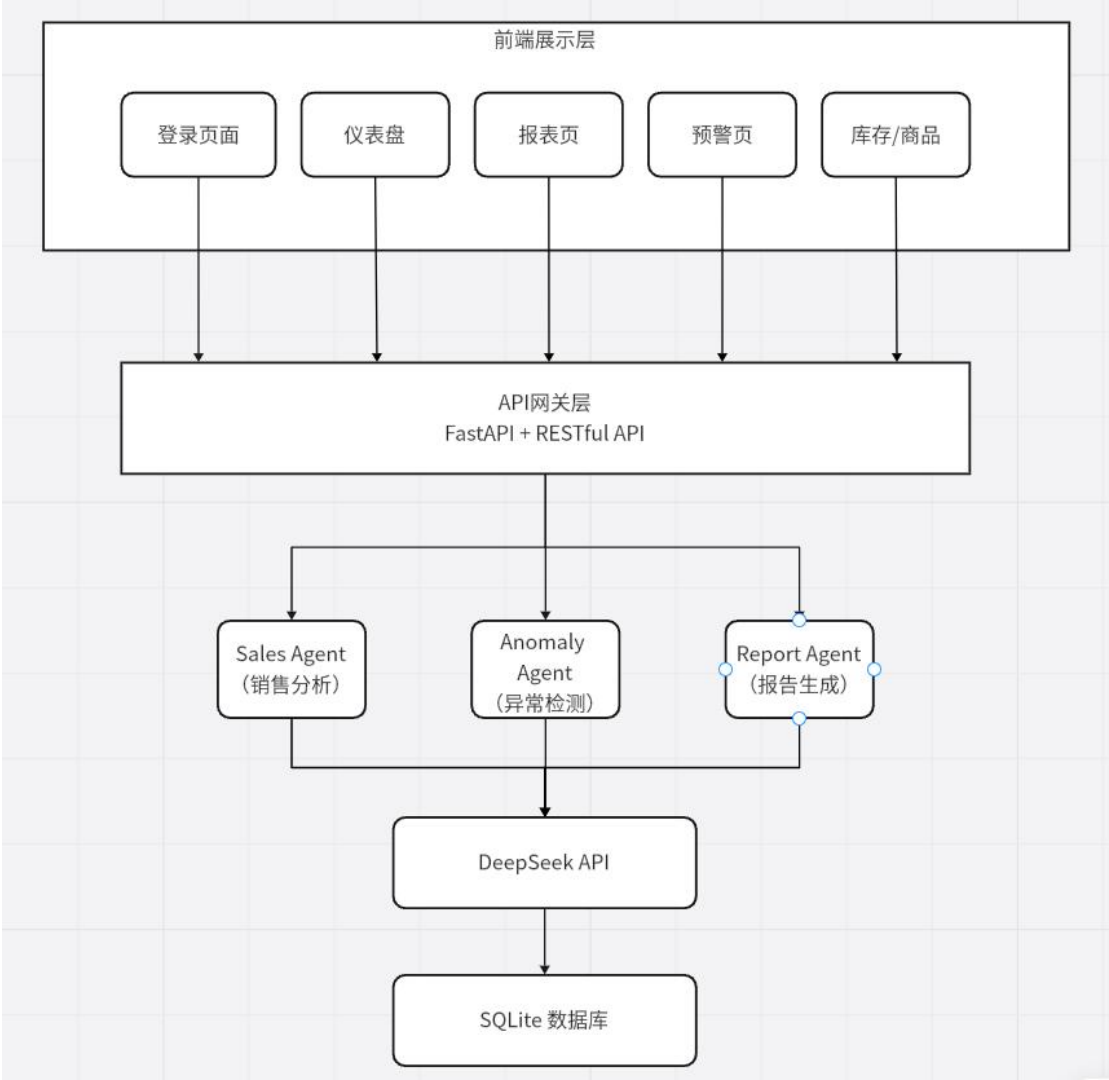


图 2.1 Architecture Spec

2.3 API Spec

系统的应用程序编程接口设计涵盖十个主要端点。其中，针对库存管理，提供 GET 方法获取库存列表、POST 方法新增库存记录、PUT 方法更新指定库存信息、DELETE 方法删除特定库存，另有 GET 接口分别用于获取库存类别列表和库存名称列表。商品管理方面提供 GET 接口获取商品列表。告警管理通过 GET 接口获取系统生成的警报记录。销售分析通过 GET 接口触发异常检测任务。报表生成采用 POST 接口触发智能报表生成流程，另有独立的 GET 接口用于获取 AI 生成的日报。最后，智能问答助手通过 POST 接口并以流式响应方式提供报表内容的交互式问答服务。

API 端点	HTTP 方法	功能描述
/api/inventory	GET	获取库存列表
/api/inventory	POST	新增库存
/api/inventory/{id}	PUT	更新库存
/api/inventory/{id}	DELETE	删除库存
/api/inventory/categories	GET	获取库存类别
/api/inventory/names	GET	获取库存名称
/api/products	GET	获取商品列表
/api/alerts	GET	获取警报列表
/api/sales/analysis	GET	执行异常检测
/api/reports/generate	POST	生成智能报表
/api/reports/daily	GET	AI 日报
/api/chat/stream	POST	AI 报表助手

三、系统架构与设计

3.1 核心架构图

系统整体采用四层架构进行构建。在用户层，客户端请求通过 FastAPI 网关进入系统，该网关承担请求路由与协议转换职责。应用层负责处理具体业务逻辑，其中智能体模块调用大语言模型完成智能分析任务，传统业务模块则直接处理常规数据操作。所有业务数据最终统一存储于 SQLite 数据库之中，各层之间形成清晰的调用关系。

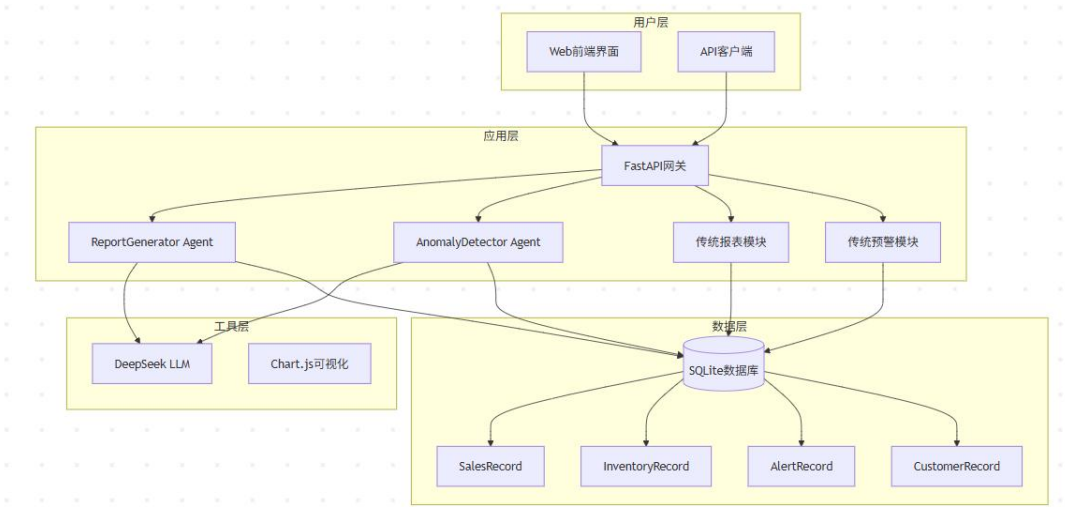


图 3.1 核心架构图

3.2 Agent 交互流程

3.2.1 报表生成 Agent 交互流程

报表生成智能体的工作流程如下：用户发起报表生成请求后，系统实例化对应的智能体对象。该智能体首先从数据库中提取所需的原始业务数据，随后对数据进行指标计算与统计分析。完成数据预处理后，智能体调用大语言模型接口，将计算结果与提示词模板一同提交，由模型生成智能化的分析文本。最后，系统将分析结果整合为结构化的报告文档并返回给用户。

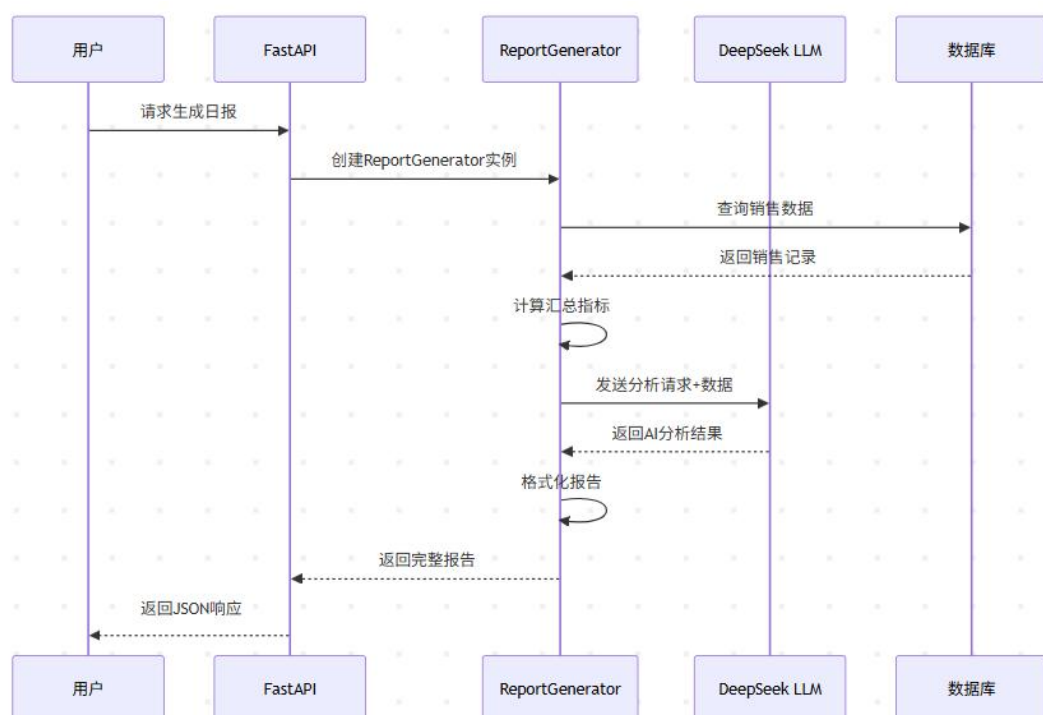


图 3.2 智能报表 Agent 工作流程

3.2.2 异常检测 Agent 交互流程

异常检测智能体的运行流程分为四个步骤。首先，智能体从数据库中获取当前的库存数据与销售数据。其次，基于预设的业务规则对数据进行扫描，识别出偏离正常范围的异常记录。随后，针对每个识别出的异常，智能体调用大语言模型生成相应的处理建议与应对策略。最后，系统将生成的告警记录持久化存储至数据库，并将增强后的检测结果返回给前端展示。

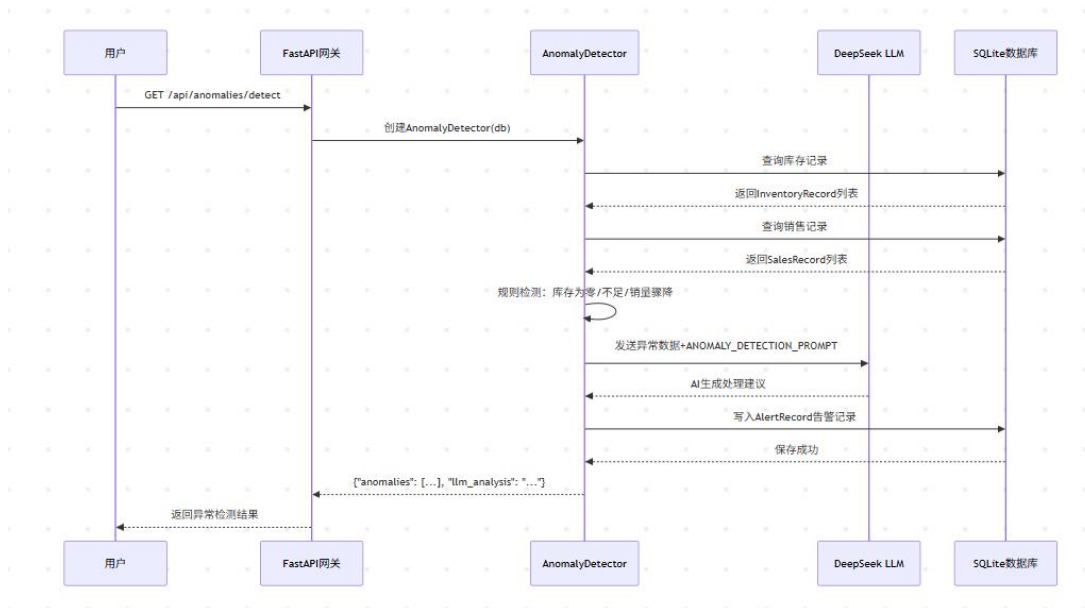


图 3.3 异常检测 Agent 工作流程

3.3 数据流设计

在数据流设计方面，原始业务数据从各个业务系统流入中央数据库。数据经过清洗、转换与指标计算等预处理环节后，分别流向智能报表生成模块和异常预警模块。两个模块的处理结果最终汇聚至可视化展示层，以图表和文本形式呈现给终端用户，形成从数据采集到价值呈现的完整数据流水线。

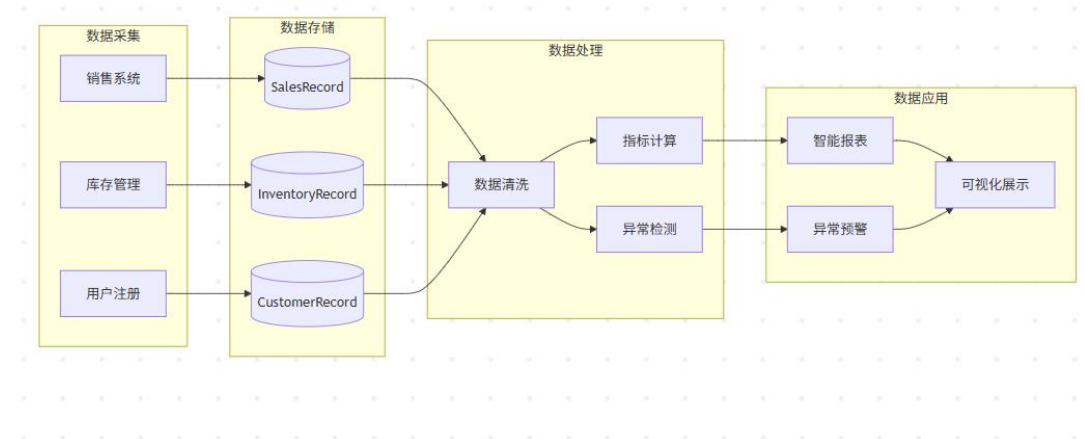


图 3.4 数据流设计

3.4 AI vs 传统对比架构

对比人工智能增强版本与传统版本的系统架构可以发现显著差异。人工智能版本通过智能体协调数据查询与大语言模型分析，生成经过智能增强的分析结果并予以持久化存储。相比之下，传统版本直接执行基于规则的判断逻辑，实时返回分析结果。人工智能版本相较于传统版本，能够减少用户在数据分析与策略制定环节的时间投入，从而整体提升运营效率。

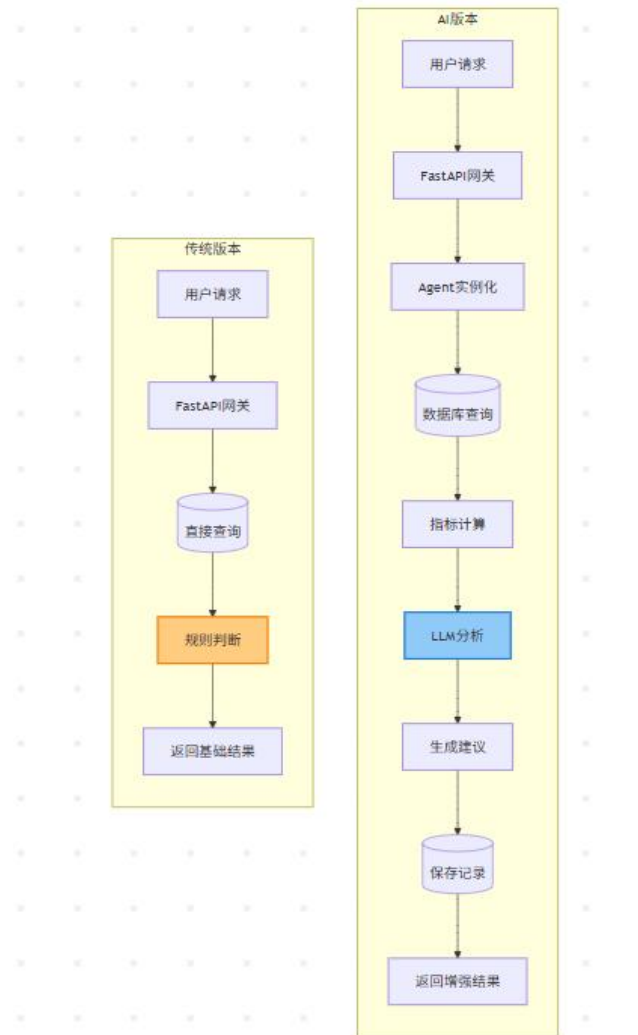


图 3.5 AI vs 传统对比架构

四、关键实现与代码展示

4.1 Agent 核心循环

4.1.1 智能报表 agent 核心逻辑

智能报表智能体的核心实现逻辑如图 4.1 所示。该实现主要包括数据获取模块、指标计算模块、大语言模型调用模块以及报告组装模块。智能体首先通过数据库查询接口获取指定时间范围内的销售明细数据，随后计算总销售额、订单量、客单价、各类别销售占比等关键业务指标。完成指标计算后，系统将计算结果填充至预设的提示词模板中，调用大语言模型接口生成自然语言形式的业务分析结论。最终，智能体将原始数据、计算指标与模型分析结果整合为统一的报告结构并返回。

```
class ReportGenerator:
    def __init__(self, db: Session):
        self.db = db
        self.llm = get_deepseek_client()

    def generate_daily_report(self):
        # 1. 数据采集
        today_sales = self.db.query(SalesRecord).filter(...).all()

        # 2. 指标计算
        total_sales = sum(s.total_amount for s in today_sales)

        # 3. LLM分析
        prompt = DAILY_REPORT_PROMPT.format(data=...)
        response = self.llm.invoke(prompt)

        # 4. 返回结果
        return {"summary": {...}, "analysis": response.content}
```

图 4.1 智能报表 agent 核心逻辑

4.1.2 异常 Agent 核心逻辑

异常检测智能体的核心逻辑如图 4.2 所示。该智能体首先定义多级异常检测规则，包括零库存检测、销量骤降检测以及库存低于安全阈值检测三类场景。在每次检测周期中，智能体遍历商品列表，逐项应用上述规则进行判定。对于触发规则的商品项，系统根据异常类型分配相应的优先级等级，并调用大语言模型针对该具体异常生成定制化的处理建议。所有检测结果经过去重与合并后，统一写入告警数据表，同时返回至调用方。

```

# src/app/agent/anomaly_detector.py
class AnomalyDetector:
    def __init__(self, db: Session):
        self.db = db
        self.llm = get_deepseek_client()

    def detect_inventory_anomalies(self):
        """检测库存异常"""
        inventory = self.db.query(InventoryRecord).all()
        anomalies = []

        for item in inventory:
            if item.stock_quantity == 0:
                anomalies.append({
                    "type": "inventory_empty",
                    "product_name": item.stock_name,
                    "severity": "high"
                })
            elif item.stock_quantity < item.threshold:
                anomalies.append({
                    "type": "inventory_low",
                    "product_name": item.stock_name,
                    "severity": "low"
                })
        return anomalies

    def generate_anomaly_report(self):
        """生成异常报告"""
        anomalies = self.detect_inventory_anomalies() + self.detect_sales_anomalies()

        if anomalies:
            prompt = ANOMALY_DETECTION_PROMPT.format(data=str(anomalies))
            response = self.llm.invoke(prompt)
            return {"anomalies": anomalies, "analysis": response.content}

```

图 4.2 异常 Agent 核心逻辑

4.2 LLM 客户端配置

大语言模型客户端的配置代码如图 4.3 所示。配置内容包括 API 基础地址、认证密钥、默认使用的模型名称、请求超时时间以及最大重试次数等参数。系统采用环境变量方式管理敏感配置项，确保部署过程中的安全性。

```

# src/app/llm/client.py
def get_deepseek_client():
    """获取DeepSeek客户端"""
    api_key = os.getenv("DEEPSEEK_API_KEY")
    base_url = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com/v1")

    return ChatOpenAI(
        api_key=api_key,
        base_url=base_url,
        model="deepseek-chat",
        temperature=0.7
    )

```

图 4.3 LLM 客户端配置代码

报表智能体的提示词模板如图 4.4 所示。该模板采用结构化设计，明确要求大语言模型基于提供的销售数据生成专业、客观的业务分析报告，涵盖整体销售概况、趋势变化解读、品类表现对比以及潜在风险提示等维度。

```
DAILY_REPORT_PROMPT = """
你是一位资深的数据分析专家，请根据以下销售数据生成专业的分析报告：

{data}

请分析：
1. 销售趋势
2. 品类表现
3. 潜在问题
4. 改进建议
"""
```

图 4.4 报表 Agent 提示词模板

异常智能体的提示词模板如图 4.5 所示。该模板要求模型针对每个检测到的异常，从可能原因分析和具体行动建议两个层面生成回复，每条建议应当具备可操作性，且每个异常最多生成三条建议。

```
# src/app/llm/prompts.py
ANOMALY_DETECTION_PROMPT = """
你是电商运营专家，请分析以下异常数据并提供处理建议：

数据：{data}

规则：
- 高优先级：库存为0
- 中优先级：销量下降超过50%
- 低优先级：库存低于阈值

要求：每个异常最多3条建议，每条≤30字
"""
```

图 4.5 异常 Agent 提示词模板

4.4 AI IDE 使用截图

商品与库存管理界面的构建过程如图 4.6 所示。界面采用表格形式展示商品与库存清单，支持行内编辑与新增操作，并提供按类别筛选的下拉选择器。

异常文本修改功能的界面如图 4.7 所示。系统允许用户对智能体生成的异常描述文本进行人工修订，修订记录将用于后续提示词的优化迭代。

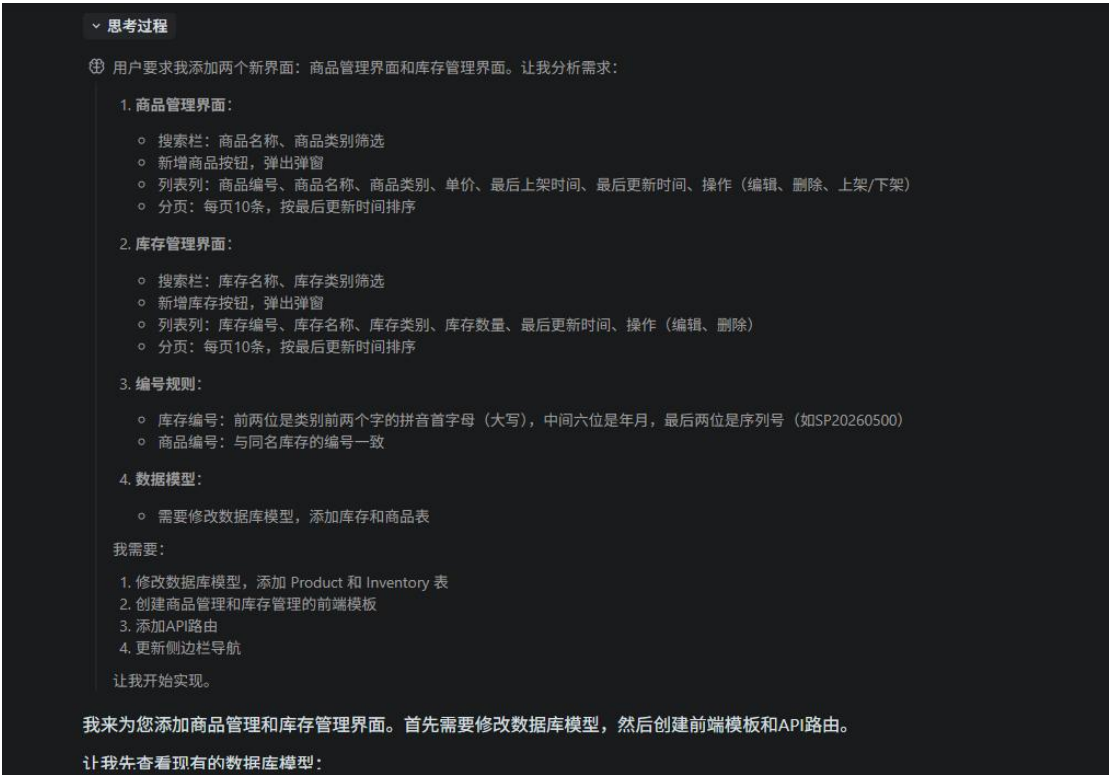


图 4.6 商品、库存管理界面构建

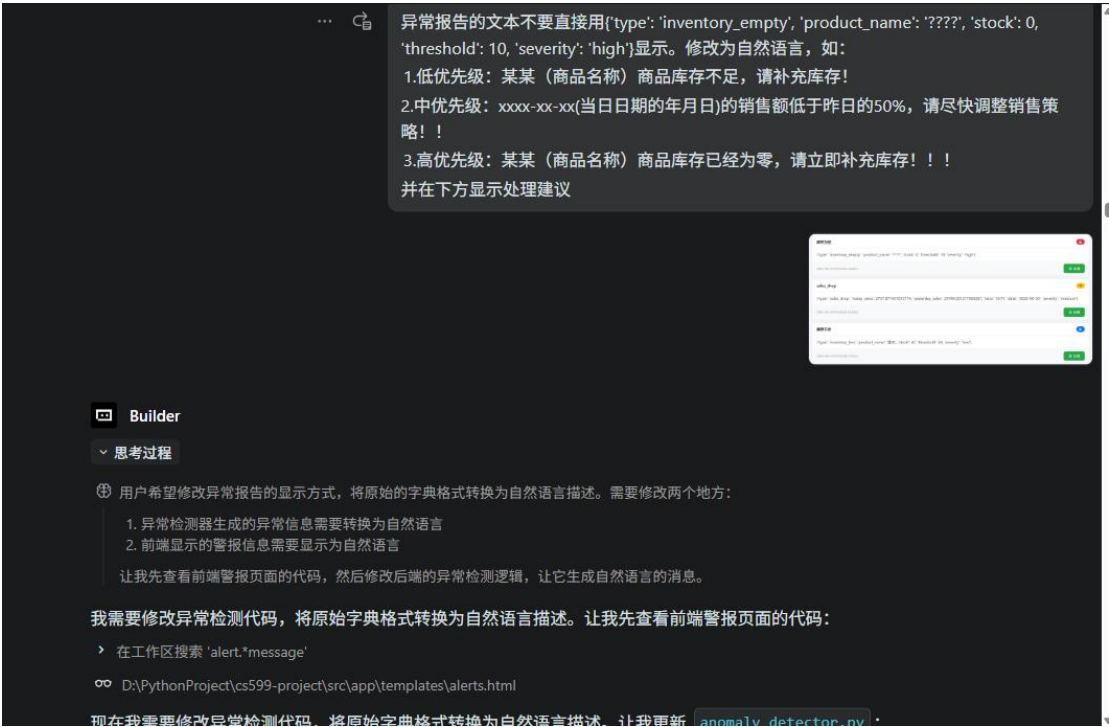


图 4.7 异常文本修改

五、测试与评估

5.1 功能测试

功能测试覆盖六个主要测试项。在库存异常检测测试中，当某商品库存数量为零时，系统能够成功触发高优先级告警，实际测试结果为通过。在销量异常检测测试中，当某商品销量环比下降幅度达到百分之五十时，系统正确生成中优先级警报，测试结果为通过。在库存不足检测测试中，当商品库存低于预设安全阈值时，系统成功触发低优先级告警，测试结果为通过。在人工智能建议生成测试中，针对每个检测到的异常，系统均能生成不超过三条且内容合理的处理建议，符合预期要求。在报表生成测试中，系统能够自动生成内容完整、逻辑连贯的自然语言报告，测试结果为通过。在报表智能助手测试中，系统可以切实感知到报表内容，符合设计要求。

测试项	测试描述	预期结果	实际结果
库存异常检测	库存为 0 时触发高优先级告警	生成高优先级警报	✔ 通过
销量异常检测	销量下降 50%触发中优先级告警	生成中优先级警报	✔ 通过
库存不足检测	库存低于阈值触发低优先级告警	生成低优先级警报	✔ 通过
AI 建议生成	每个异常生成最多 3 条建议	建议符合要求	✔ 通过
报表生成	自动生成自然语言报告	报告内容完整	✔ 通过
报表智能助手	结合报表回答问题	正确感知报表	✔ 通过

5.2 Agent 行为评估

智能体在实际运行中的行为表现表明，报表生成智能体能够根据不同的数据分布特征生成差异化的分析结论，而非简单套用固定模板。异常检测智能体在生成告警建议时表现出较好的上下文感知能力，所提建议与具体异常类型高度相关。其次，报表智能助手也能正确感知报表内容。可以针对用户提出的问题，结合报表内容进行回答。

5.3 Demo 截图说明



图 5.1 自然语言报表



图 5.2 报表智能助手

与之相比，传统报表虽然也可以直观显示数据指标，但是理解和总结出需要的信息和策略仍会花费大量时间，在整体效率层面，弱于智能报表。



图 5.3 传统报表界面



图 5.4 数据可视化

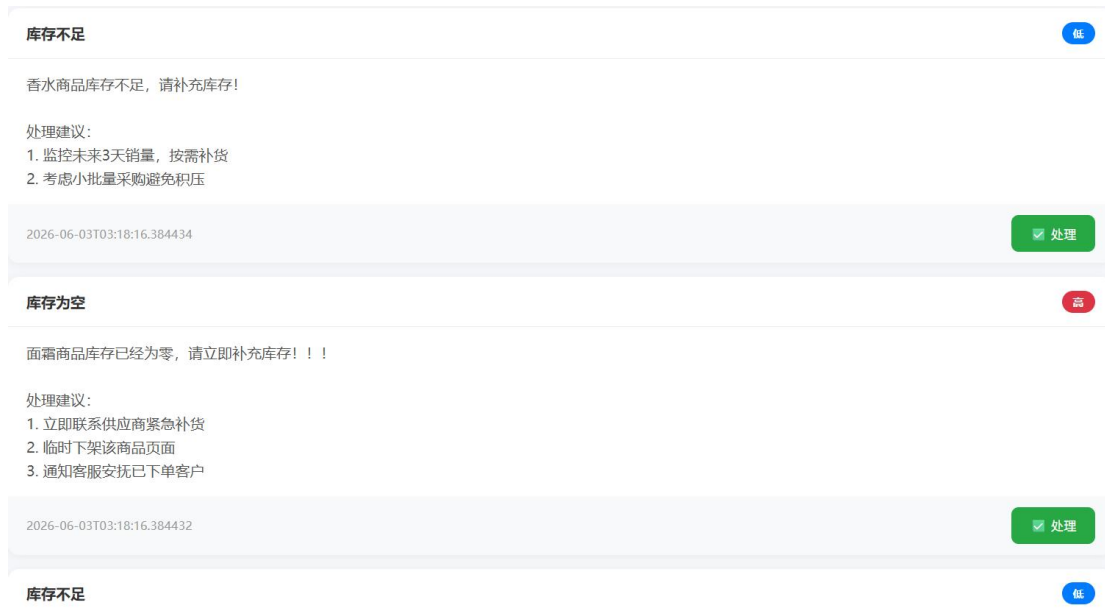


图 5.5 异常智能预警

与之相比，传统预警虽然也可以根据数据做出正确的预警，但是理解和总结出需要的信息和策略仍会花费大量时间，在整体效率层面，弱于异常智能预警。

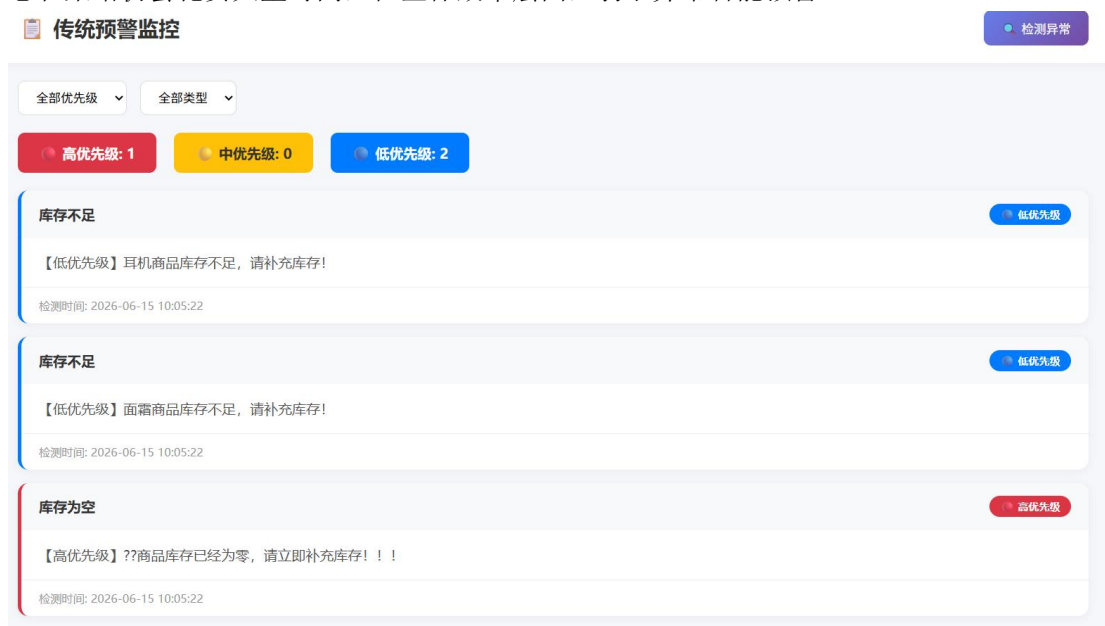


图 5.6 传统异常预警

六、系统升级与扩展

6.1 可扩展架构设计

系统采用的可扩展架构如图 6.1 所示。该架构在智能体层设计了标准化的接口规范，新增业务功能只需实现相应接口即可无缝接入现有框架。数据访问层采用数据访问对象模式，底层数据库的更换不会影响上层业务逻辑。大语言模型调用层采用适配器模式，便于后续替换或接入不同厂商提供的模型服务。

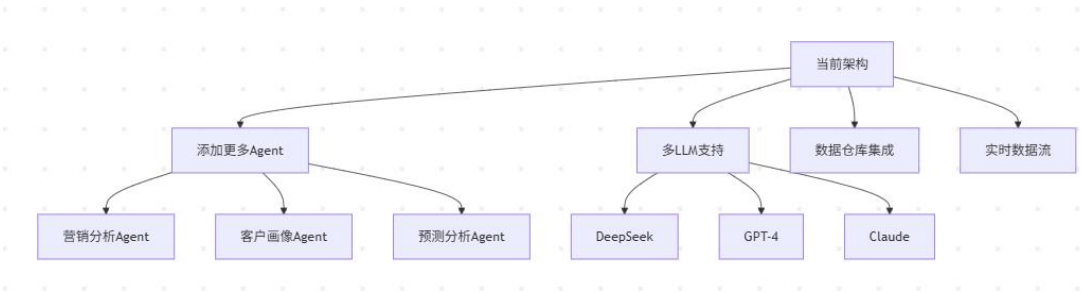


图 6.1 可扩展架构

6.2 下一阶段计划

下一阶段的开发工作包含四个主要方向。第一，实现多数据源接入能力，支持从企业资源计划系统、客户关系管理系统等外部业务系统导入数据。第二，集成机器学习模型，训练针对特定业务场景的定制化异常检测模型，以补充规则引擎的不足。第三，建立实时预警推送机制，支持通过电子邮件、短信、即时通讯工具等多种渠道向用户发送告警通知。第四，完成移动端适配，开发面向手机和平板设备的响应式 H5 页面。

6.3 AI 能力演进路径

人工智能能力的演进路径划分为三个阶段。当前阶段采用规则引擎与大语言模型分析相结合的方式，以规则检测发现异常，以大语言模型生成分析建议。中期目标是在此基础上集成自定义机器学习模型，提升异常检测的准确率与召回率。长期目标是实现系统的自适应学习能力，使系统能够根据用户反馈与业务变化自动调整检测规则与决策建议，最终形成具备持续进化能力的智能决策系统。

七、课程总结

7.1 个人收获

通过本项目的实践，我在以下四个方面获得了显著提升。首先，在智能体技术理解方面，深入掌握了智能体的核心概念，包括感知环境信息、做出决策判断以及执行相应动作的完整闭环流程。其次，在大语言模型应用实践方面，掌握了如何将大语言模型能力集成到实际业务系统之中，实现自然语言交互驱动的数据分析功能。再次，在系统架构设计方面，学会了设计具备良好可扩展性的系统架构，为后续功能迭代与性能优化预留了充分空间。最后，在全栈开发能力方面，从前端界面构建到后端服务开发再到数据库设计，整体开发能力得到了系统性的锻炼与提升。

7.2 工程思维转变

本项目的实施促使我完成了三个层面的工程思维转变。第一，从功能驱动思维转向数据驱动思维，学会了用数据而非直觉来驱动业务决策过程。第二，从被动响应思维转向主动预测思维，构建了主动发现潜在问题而非等待问题暴露后再处理的能力。第三，从固定逻辑思维转向自适应思维，认识到可以利用人工智能技术实现灵活可变的业务规则，使系统能够适应不断变化的业务环境。

7.3 对课程的建议

基于本项目的学习体验，对课程建设提出三点建议。第一，建议增加更多贴近真实商业场景的实践案例，使学生在课程学习过程中能够接触到更丰富的业务问题类型。第二，建议强化部署实践环节的教学内容，增加关于云平台部署、容器化封装以及持续集成与持续部署流程的讲解与练习。第三，建议在课程中介绍更多主流大语言模型的使用方法与应用模式，帮助学生建立更全面的模型选型视野。